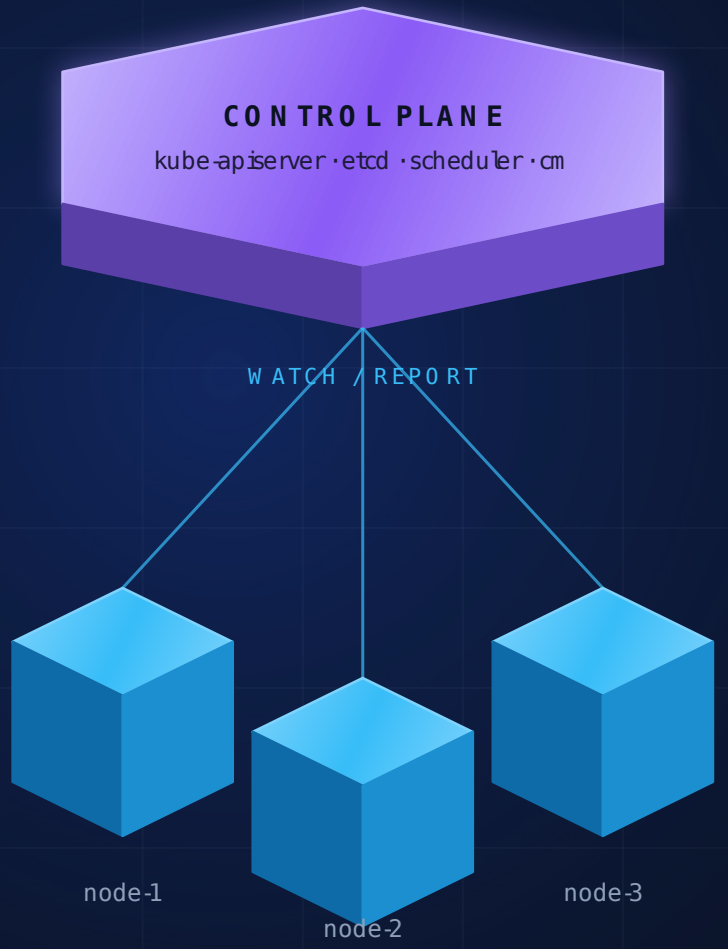
CLUSTER ARCHITECTURE · KUBERNETES.IO DOCS · 2026

Kubernetes
集群架构拆解

两个平面、一条脊柱：
从控制平面到 Pod 的每一层组件与它们如何对话



资料来源： kubernetes.io 官方文档（Cluster Architecture / Components / Service / Pod Lifecycle / Persistent Volumes）， 2026-05 整理

两个平面，一条脊柱

一个 Kubernetes 集群是两平面系统；所有组件都只通过 kube-apiserver 交流，这是整座架构的脊柱。

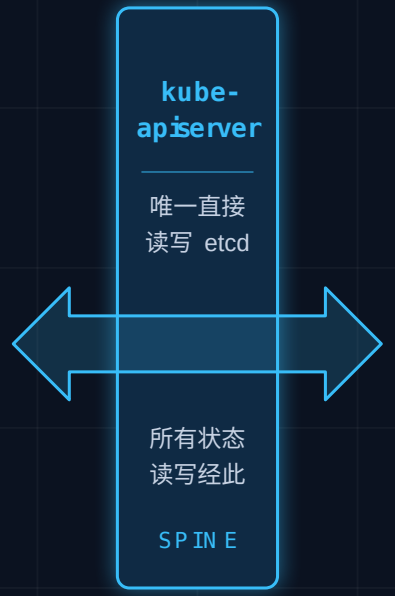
控制平面

CONTROL PLANE

全局决策者。

- 调度工作负载到节点
- 响应集群事件
- 保存每个 API 对象的期望状态

最小集群：1 台控制平面主机 + 1 个工作节点



数据平面

DATA PLANE · WORKER NODES

真正运行应用 Pod 的地方。

- 一个或多个工作节点
- 每个节点： kubelet + 容器运行时 + kube-proxy
- 节点池规模按工作负载确定

生产：控制平面与 etcd 跨 ⩾ 3 台主机、跨可用区分布

≥ 3 台控制平面主机
生产环境； kube-apiserver 置于负载均衡之后

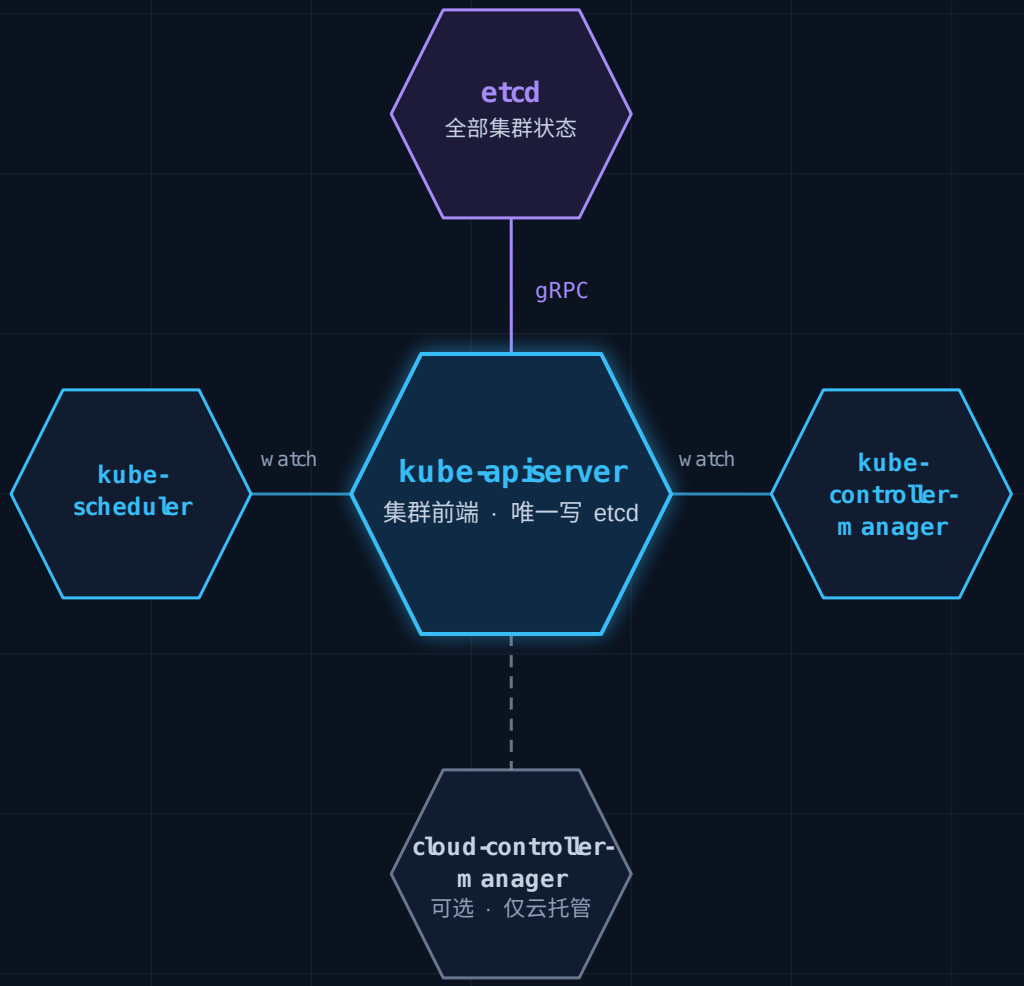
3/5 个 etcd 成员（奇数）
Raft 共识需要奇数成员形成法定人数

1 个组件直接对话 etcd
其余组件都经由 API server 读写状态

接下来：先拆控制平面的五个组件，再看每个工作节点上的三个组件。

控制平面的五个组件

四个组件围着 API server 转； etcd 只和它说话。



kube-apiserver

集群前端，暴露 Kubernetes HTTP API ； 校验、准入后再持久化。
唯一直接读写 etcd 的组件；可水平扩展（多实例 + 负载均衡）。

etcd

分布式、一致、高可用的 KV 存储，保存全部集群状态。
Raft 共识，奇数成员（3 或 5）；丢 etcd 即丢集群，备份是命脉。

kube-scheduler

监听还没有 nodeName 的 Pod ，为它挑一个节点。
只把绑定决定写回 API server ，从不直接和节点对话。

kube-controller-manager

一个二进制、多个逻辑控制器，各自跑 watch → diff → act 循环。
Leader 选举保证高可用：同一时刻只有一个副本活跃。

cloud-controller-manager 可选

仅云托管集群有；封装厂商逻辑，让核心二进制保持厂商中立。
子控制器： Node / Route / Service （云负载均衡）。

调度：先过滤，再打分

kube-scheduler 为每个还没有 nodeName 的 Pod 走一条两阶段流水线，结果只是一条绑定记录。

STAGE 1 · FILTERING

过滤 —— 哪些节点是可行的?

STAGE 2 · SCORING

打分 —— 哪个可行节点最优?



过滤依据

- 资源请求 CPU / memory / GPU
- 硬件 / 软件 / 策略约束
- 亲和与反亲和规则 affinity / anti-affinity
- 污点与容忍 taints / tolerations
- 拓扑分布、数据局部性、工作负载间干扰

任何一条不满足，节点就被淘汰，不进入打分。

打分与绑定

对每个可行节点按同一组规则打分，取最高分者。

调度器随后做的只有一件事：把 binding 写回 API server。
真正拉起容器的是那个节点上的 kubelet —— 调度器不直接和节点对话。

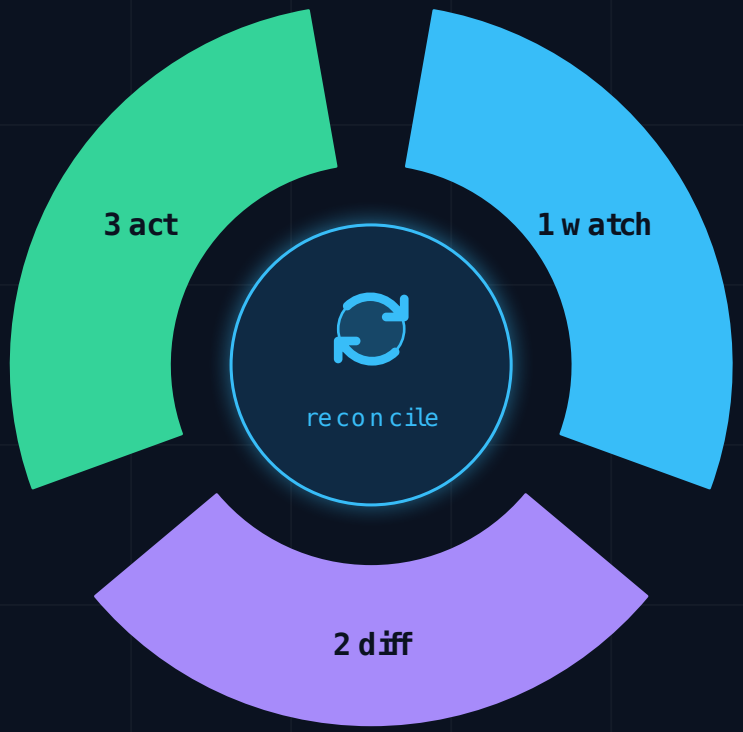
一个 Pod 一生只被调度一次

失败的 Pod 不会被“重新调度”，而是由它的控制器（Deployment / StatefulSet / Job）用一个新的 UID 创建替代 Pod，再走一遍完整的过滤 → 打分流水线。

调度器决定“去哪”；接下来看控制器如何持续保证“数量与状态对”。

控制器：一个永不停止的调和循环

kube-controller-manager 里的每个控制器都在做同一件事：让实际状态不断逼近期望状态。



顺时针循环，永不退出

1 · watch

通过 API server 监听自己关心的对象（Pod、Node、Service…）

2 · diff

比较对象的期望状态（spec）与实际状态（status）

3 · act

执行动作缩小差距：创建 Pod、更新端点、标记节点…然后回到 1

核心控制器

Node	检测并响应节点故障
Job	创建 Pod 驱动 Job 对象跑到完成
EndpointSlice	把 Service 映射到 Pod IP
ServiceAccount	为新命名空间创建默认 ServiceAccount
Workload 系列	ReplicaSet / Deployment / StatefulSet / DaemonSet 等工作负载原语

高可用靠 Leader 选举：多副本部署，但同一时刻只有一个副本在真正动手。

每个工作节点上的三个组件

节点代理、 Service 实现、容器运行时——三者并列， kubelet 是唯一和 API server 对话的那个。

WORKER NODE · ×N

kubelet

节点级代理

- 监听 API server 上调度到本节点的 Pod
- 拿到 PodSpec ，指挥运行时起容器
- 用探针监督健康，回报节点与 Pod 状态
- 容器死了按 restartPolicy 重启：

`Always` / `OnFailure` / `Never`

- 通过 kube-node-lease 里的 Lease 心跳

持续调和：只要和期望不一致就动手

kube-proxy

节点级 Service 实现 · 可选

- 维护网络规则： Service 虚拟 IP → Pod IP
- 四种后端：

`iptables` 默认， netfilter ， CPU 开销低

`IPVS` 内核 L4 负载均衡，海量 Service 更佳

`nftables` 现代 netfilter ，较新集群

`userspace` 旧实现，慢，已废弃

• Cilium 、 Calico-eBPF 等 CNI 插件
可以整体替代 kube-proxy

它实现的是抽象，不是转发本身

container runtime

真正跑容器的组件

• CRI （Container Runtime Interface ）
标准化 kubelet 与运行时的契约

- 支持的运行时：

`containerd`

`CRI-O`

任何 CRI 兼容实现

Docker Engine 自 `v1.24` 起
不再作为内置运行时

kubelet 只认 CRI ，不认具体实现

有了节点上的三件套，就可以看 Pod 在节点上从生到死的完整过程。

Pod 的一生：阶段、探针与优雅终止

Pod = 共享网络命名空间与存储卷的一组容器，最小可部署单元。



Pending — 已接受，容器未全部就绪；等调度、拉镜像等

Running — 已绑定节点，≥ 1 容器在运行 / 启动 / 重启中

Succeeded — 全部容器退出码 0；不会再重启

Failed — 全部容器已退出；≥1 个非零退出

Unknown — API server 联系不上持有该 Pod 的节点

阶段由控制平面设置； Pod 内每个容器另行追踪自己的状态：`Waiting` / `Running` / `Terminated`

探针 · 由 kubelet 执行

livenessProbe
失败 ⇒ kubelet 重启该容器

readinessProbe
失败 ⇒ 从 Service 端点移除，不重启

startupProbe
应用初始化完成前，挡住前两种探针

探测方式 `exec` / `httpGet` / `tcpSocket` / `grpc`

调度只发生一次：失败的 Pod 由控制器用新 UID 替换，而非原地重调度。

优雅终止 · 默认宽限 30s

1. API server 把 Pod 标记为待删除
2. `preStop` 钩子（若定义）在容器内运行
3. 向容器主进程发送 `SIGTERM`
4. 最多等待 `terminationGracePeriodSeconds` 让它自行退出
5. 仍活着 ⇒ `SIGKILL`

把清理逻辑放进 preStop 和 SIGTERM 处理，别指望第 5 步。

Service：多变的 Pod 前面，一个稳定的入口

一个 Service 给不断变化的一组 Pod 提供稳定的虚拟 IP 和 DNS 名；四种类型是层层向外包裹的关系。

ClusterIP

DEFAULT

可达范围

仅集群内部

实现

Service CIDR 里的虚拟 IP；kube-proxy 编程 iptables / IPVS 规则做转发

NodePort

STATIC PORT

可达范围

任一节点 IP 的静态端口

30000–32767

实现

每个节点都打开该端口，再路由到 ClusterIP

LoadBalancer

CLOUD LB

可达范围

外部云负载均衡器

实现

cloud-controller-manager 供给 LB，后端指向 NodePort

ExternalName

DNS CNAME

可达范围

DNS 别名，不在集群内代理

实现

API server 直接返回 CNAME 记录，没有转发

箭头方向 = 依赖：LoadBalancer 建立在 NodePort 之上，NodePort 建立在 ClusterIP 之上；ExternalName 独立于三者。

EndpointSlices

追踪 Service 背后的 Pod IP。

v1.21 起替代旧的 Endpoints；

控制器随 Pod 来去持续重建切片。

ClusterDNS

`<svc>.<namespace>.svc.cluster.local`

每个 Service 都可按此名解析；

CoreDNS 是生产集群的必装插件。

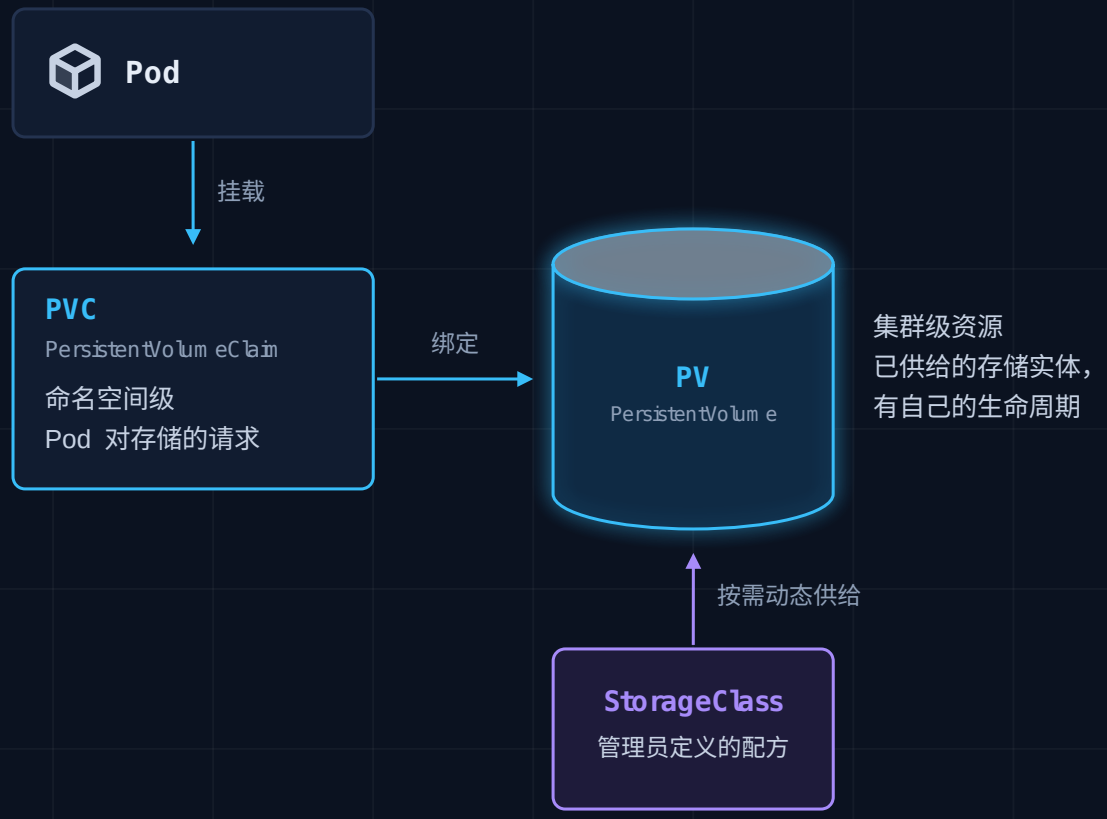
Headless Service

`clusterIP: None`

不分配虚拟 IP，DNS A 记录直接返回 Pod IP；StatefulSet 靠它让每个副本拥有自己可解析的名字。

存储：三种资源分工，CSI 统一后端

Pod 只提出请求（PVC），管理员定义配方（StorageClass），集群持有实体（PV）。



CSI 驱动是现代插件契约：每种后端自带一个驱动，树内插件已废弃。

访问模式 · accessModes

ReadWriteOnce	RWO	单个节点可读写挂载
ReadOnlyMany	ROX	多个节点只读挂载
ReadWriteMany	RWX	多个节点可读写挂载
ReadWriteOncePod	RWOP	仅单个 Pod 可读写

“Once / Many”说的是节点数，“OncePod”才是 Pod 级独占。

回收策略 · PVC 删除后 PV 怎么办

Retain　Delete　Recycle · 废弃

卷模式 · volumeMode

Filesystem　默认，挂载为目录　　Block　裸块设备

生产高可用：三台起步，奇数 etcd ，跨可用区

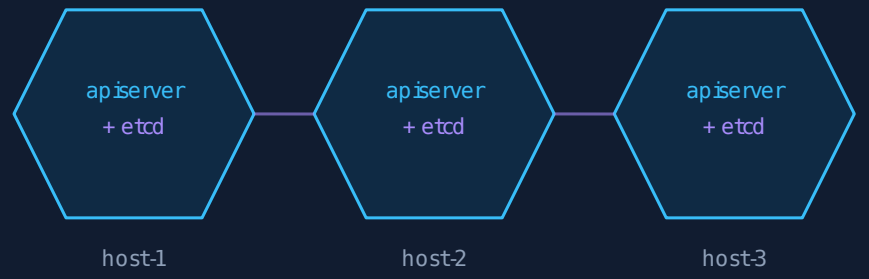
- **≥3 台控制平面主机**
  kube-apiserver 置于负载均衡后
- **scheduler / cm Leader 选举**
  多副本，同一时刻一个活跃
- **etcd ≥3 成员，取奇数**
  3 或 5，跨主机分布以形成法定人数
- **跨可用区分布**
  控制平面与工作节点都跨 AZ；丢一个区，集群继续跑

控制平面有两种拓扑可选，差别只在 etcd 放哪。

TOPOLOGY A

Stacked · 堆叠式

每台控制平面主机同时跑一个 etcd 成员和 kube-apiserver



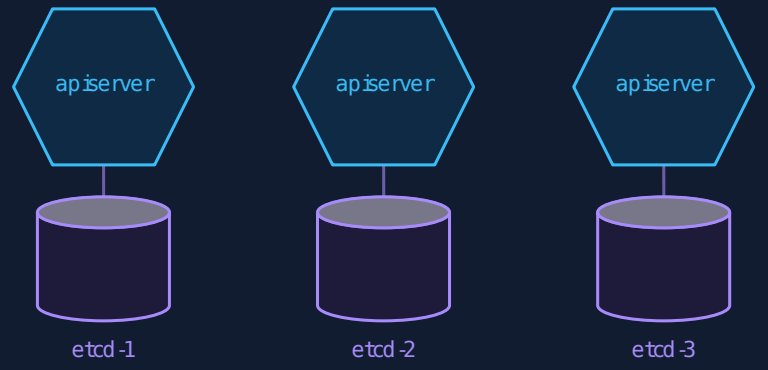
+ 更简单，硬件更少

− 一台主机故障 = 同时损失一个 apiserver 和一个 etcd 成员

TOPOLOGY B

External etcd · 外置式

etcd 集群跑在专用主机上，控制平面主机只跑 apiserver 等



+ 更好的故障隔离：apiserver 与 etcd 各自独立倒下

− 需要更多硬件

自愈的边界：什么会自动恢复，什么不会

会自动恢复

容器崩溃 → kubelet 按 restartPolicy 重启，指数退避
100ms → 5min 上限，健康运行 10min 后重置

Pod 失败 → 所属控制器（Deployment / ReplicaSet / StatefulSet）创建新 Pod

节点失败 → 节点控制器标记 NotReady；超过 pod-eviction-timeout（默认 5min）后其上 Pod 被标记删除，控制器在其他节点重建

负载不均 → kube-scheduler 把新 Pod 放到负载更低的节点

共同点：都有一个控制器在盯着，差距一出现就动手。

不会自动恢复

etcd 失去法定人数

集群状态随之丢失；只能从备份手动恢复——这就是为什么备份是命脉。

API server 失联

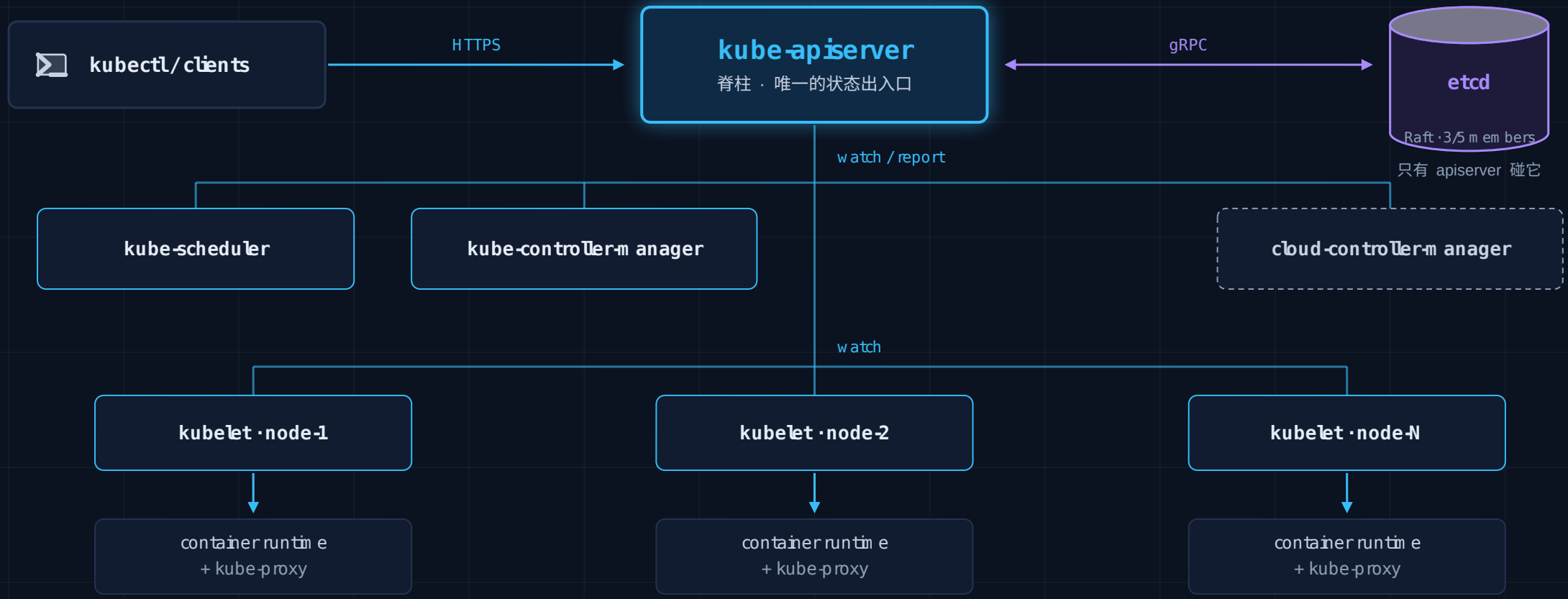
无法再做任何新决策；已有工作负载继续在各节点运行，直到 kubelet 的 watch 超时为止。

脊柱断了，循环就停了。

生产必备的可观测插件

- **DNS** 必需 — CoreDNS
- **指标** — metrics-server + Prometheus
- **日志** — FluentBit/Fluentd → Loki·Elasticsearch · 云日志
- **Dashboard** 可选 — Web UI

通信总览：每一条线都经过 API server



没有任何组件直接对话另一个组件——一切经由 API server；也只有 API server 对话 etcd。

于是每一次状态变化都是一个 API 事件：这正是 Kubernetes 可插拔、可观测的根源。

回顾：五句话记住这套架构

01 **两个平面**

控制平面做决策，工作节点跑 Pod；最小 1+1，生产 ≥ 3 台控制平面主机。

02 **一条脊柱**

所有组件只和 kube-apiserver 说话；只有它读写 etcd。每次状态变化都是一个 API 事件。

03 **调度一次，调和不停**

scheduler 过滤再打分、只写绑定；之后由控制器的 watch → diff → act 循环维持期望状态。

04 **稳定入口与分工存储**

Service 给多变的 Pod 一个稳定的 IP 和 DNS 名； PVC / PV / StorageClass 把请求、实体、配方分开， CSI 统一后端。

05 **自愈有边界**

≥3 控制平面 + 奇数 etcd + 跨可用区能扛住大多数故障； etcd 法定人数一丢，只有备份能救。

SOURCES · KUBERNETES.IO

Cluster Architecture · Kubernetes Components · Service · Pod Lifecycle · Persistent Volumes （官方文档， 2026-05 整理）

END OF DECK